m	*Multiline mode.*	Normally, "." matches any regexp option character except a newline. With the /m option, "." matches any character.
x	*Extended mode.*	Complex regular expressions can be difficult to read. The x option allows you to insert spaces and newlines in the pattern to make it more readable. You can also use # to introduce comments.

It's worth taking a second to explore extended mode, which allows you to add whitespace and comments into the regular expression definition. If there's actually whitespace in the pattern, you need to explicitly use character classes to denote the whitespace. This can make the regular expression more readable – we'll see what all this syntax means in a moment:

```
city_state_zip = %r{
  (\w.*),        # city name followed by a comma
  \s               # a space
  ([A-Z][A-Z])  # a two character state abbreviation
  \s               # a space
  (\d{5})          # 5 digits for the US simple zip code
}x

"Chicago, IL 60601".match?(city_state_zip) # => true
```

Another set of options allows you to set the language encoding of the regular expression. If none of these options is specified, the regular expression will have US-ASCII encoding if it contains only 7-bit characters. Otherwise, it will use the default encoding of the source file containing the literal. The options are:

- n: no encoding (ASCII)
- e: EUC
- s: SJIS
- u: UTF-8

Matching Against Patterns

Once you have a regular expression object, you can match it against a string using the Regexp#match(*string*) method, the boolean match? method, or the match operators =~ (positive match) and !~ (negative match). The methods and match operators are defined for both String and Regexp objects. One operand of the match operator must be a regular expression.

```
name = "Fats Waller"
name =~ /a/                  # => 1
name =~ /z/                  # => nil
/a/ =~ name                  # => 1
/a/.match(name)              # => #<MatchData "a">
Regexp.new("all").match(name) # => #<MatchData "all">
```

The different versions return different results:

Method or operator	Return on match	Return on no match
=~	index of beginning of first match in string	nil
!~	false	true
match	MatchData object	nil
match?	true	false